

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Т.В. Семененко
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Представление данных. Арифметико-
логические операции

по курсу: Архитектура ЭВМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Алгоритм программы.....	4
4 Текст программы с комментариями.....	5
5 Результат трассировки программы.....	8
6 Вывод.....	9

1 Цель работы

Целью данной лабораторной работы является изучение принципов представления данных и выполнения арифметико-логических операций на языке ассемблера для процессора Intel 8086.

2 Задание

В ходе выполнения необходимо разработать и отладить программу, реализующую заданный алгоритм обработки данных, а также выполнить проверку правильности вычислений по результатам изменения содержимого регистров и области памяти.

Исходные данные задаются в виде четырёх переменных X_1 , X_2 , X_3 и X_4 , вычисляемых по формулам

$$X_1 = N_B * (-1)^{N_B},$$

$$X_2 = (-1)^{(N_B + 1)} * (N_G * N_B),$$

$$X_3 = (-1)^{(N_B + 2)} * (N_G * N_B + N_G),$$

$X_4 = (-1)^{(N_B + 3)} * N_G$, где N_B – номер варианта, а N_G – номер группы.

Для варианта 1 и группы 4321 получаются значения $X_1 = -1$, $X_2 = 21$, $X_3 = -42$, $X_4 = 21$. В соответствии с индивидуальным заданием требуется составить программу, выполняющую последовательность операций: вычисление $R_1 = X_1 + X_2$, вычисление $R_2 = X_3 * X_2$ с учётом знака, выполнение логической операции $R_3 = X_2 \text{ AND } X_4$, запись значения X_2 в стек с помощью команды PUSH, вычисление $R_4 = X_3 / X_1$ как знакового деления и снятие значения из стека с помощью команды POP. После выполнения всех операций необходимо вывести состояние регистров и области памяти, чтобы убедиться в правильности работы программы.

3 Алгоритм программы

Алгоритм работы программы можно описать следующим образом. После запуска выполняется инициализация сегмента данных и вывод исходного состояния памяти и регистров для последующего сравнения. Далее программа поочерёдно выполняет арифметические и логические операции над исходными переменными X1–X4, записывая результаты в соответствующие ячейки памяти R1–R4.

На первом этапе выполняется сложение X1 и X2: значения из памяти загружаются в регистры AL и BL, затем производится операция ADD, после чего результат записывается в ячейку R1. На втором этапе вычисляется произведение X3 и X2 как знаковое умножение. В регистр AL загружается X3, в BL — X2, выполняется команда IMUL, которая сохраняет 16-битный результат в регистре AX. Содержимое регистра AX записывается в память по адресу R2. На третьем этапе выполняется побитовое логическое «И» (AND) между X2 и X4. В результате в регистр AL заносится значение X2, затем выполняется операция AND с содержимым X4, а результат сохраняется в ячейке R3.

Далее программа помещает значение X2 в стек: для этого в регистр AX загружается ноль в старший байт (AH=0) и значение X2 в младший (AL), после чего выполняется команда PUSH AX, записывающая его на вершину стека. На заключительном этапе выполняется знаковое деление X3 на X1. Командой CBW значение в AL расширяется до AX с учётом знака, затем выполняется команда IDIV, делящая AX на X1. Частное сохраняется в AL и затем записывается в ячейку R4. После этого программа снимает значение со стека командой POP AX, чтобы восстановить стек, выводит регистры и содержимое памяти для контроля результатов и завершает работу командой HLT.

4 Текст программы с комментариями

Код для emu8086 реализуется следующим образом:

; Реализует:

; R1 = X1 + X2

; R2 = X3 * X2 (signed)

; R3 = X2 AND X4

; PUSH X2

; R4 = X3 / X1 (signed)

; Всё в одном сегменте

SET 0

; Исходные данные (в байтах: используются младшие байты 16-битных значений)

X1: DB 0xFF ; -1 (1 байт)

X2: DB 0x15 ; 21 (1 байт)

X3: DB 0xD6 ; -42 (1 байт)

X4: DB 0x15 ; 21 (1 байт)

; Результаты

R1: DB 0x00 ; 1 байт

R2: DW 0x0000 ; 2 байта, так как результат умножения может превышать 1 байт

R3: DB 0x00 ; 1 байт

R4: DB 0x00 ; 1 байт

; Отладочный вывод: показать память сегмента 0 (первые 16 байт)

start:

print mem 0:16

print reg

; ----- R1 = X1 + X2 -----

MOV SI, OFFSET X1 ; SI -> X1

MOV DI, OFFSET X2 ; DI -> X2

MOV AL, byte DS[SI] ; AL = X1

MOV BL, byte DS[DI] ; BL = X2

ADD AL, BL ; AL = X1 + X2

MOV byte DS[OFFSET R1], AL

; ----- R2 = X3 * X2 (signed) -----

MOV SI, OFFSET X3

MOV DI, OFFSET X2

MOV AL, byte DS[SI] ; AL = X3

MOV BL, byte DS[DI] ; BL = X2

IMUL BL ; AX = AL * BL (signed)

MOV word DS[OFFSET R2], AX

; ----- R3 = X2 AND X4 -----

MOV SI, OFFSET X2

MOV DI, OFFSET X4

MOV AL, byte DS[SI]

AND AL, byte DS[DI]

MOV byte DS[OFFSET R3], AL

; ----- PUSH X2 (word) -----

MOV AH, 0

MOV AL, byte DS[OFFSET X2]

PUSH AX

; ----- R4 = X3 / X1 (signed) -----

```
MOV AL, byte DS[OFFSET X3] ; AL = X3
CBW                ; sign-extend AL->AX
IDIV byte DS[OFFSET X1] ; AX / X1 (signed) -> AL quotient, AH
remainder
MOV byte DS[OFFSET R4], AL

; очистим стек
POP AX

; Вывести регистры и память (чтобы увидеть результаты)
print reg
print mem 0:16

HLT
```


5 Результат трассировки программы

Для проверки результатов вычислений в эмуляторе выполнены следующие ручные расчёты: для операции $R1 = X1 + X2$: $0xFF$ ($X1 = -1$) + $0x15$ ($X2 = 21$) = $0x14$ (20), что совпадает с содержимым ячейки $R1$ ($0x14$); для операции $R2 = X3 * X2$ (знаковое): $(-42) * 21 = -882$, что в 16-битном дополнительном коде даёт $0xFC8E$, что соответствует записи в памяти (младший байт $R2 = 0x8E$, старший байт $R2+1 = 0xFC$); для операции $R3 = X2 \text{ AND } X4$: $0x15 \text{ AND } 0x15 = 0x15$, что совпадает со значением в $R3$; для операции $R4 = X3 / X1$ (знаковое): $(-42) / (-1) = 42$, что в шестнадцатеричной системе равно $0x2A$, что соответствует содержимому $R4$. Все ручные вычисления подтверждают корректность работы программы в эмуляторе.

На рисунке 1 изображен результат трассировки программы.

Reg	H	L	Segments		Pointers	
A	00	15	SS	0000	SP	0000
B	00	15	DS	0000	BP	0000
C	00	00	ES	0000	SI	0001
D	00	00			DI	0003

Flags:								
OF	DF	IF	TF	SF	ZF	AF	PF	CF
0	0	0	0	0	0	1	0	0

Memory															Start Address		SET
															00000		
ff	15	d6	15	14	8e	11	15	2a	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Рисунок 1 – Результат работы программы emu0886

6 Вывод

В результате выполнения лабораторной работы была разработана и отлажена программа на языке ассемблера для процессора Intel 8086, реализующая заданный алгоритм обработки данных. Программа выполняет последовательность арифметических и логических операций над исходными переменными X1–X4, результаты которых записываются в ячейки памяти R1–R4. В ходе работы были выполнены операции сложения, знакового умножения, побитового логического «И», помещение данных в стек и знакового деления. Трассировка программы показала корректное изменение содержимого памяти и регистров: вычисленные значения совпали с ожидаемыми теоретическими результатами. Это подтверждает правильность построенного алгоритма, корректность обращения к памяти и регистрам, а также правильное использование команд IMUL, IDIV, AND, PUSH и POP. Таким образом, поставленная цель достигнута, программа функционирует верно и соответствует варианту задания из методических указаний.